AttackStory — Architecture & Build Spec

Oct 30, 2026 · @klath

Summary

AttackStory reads the organisers' 15–50 GB log dataset in a single streaming pass, finds the planted attacks, labels every step with an ATT&CK technique and kill-chain stage, links the steps into paths across hosts, users and time, and shows what each attacker could reach next. It is scored on accuracy against their ground truth, on speed and memory, and on a small API; a polished UI is not required.

One-line definition: a logs-to-incidents engine that detects, labels, correlates and explains attacks in very large raw log sets on a normal laptop, with every claim linked to the log line that proves it.

Layer	Who owns it	In the competition
Log dataset	Organisers	100 MB dev sample at kickoff; full tier (15, 30 or 50 GB) at build start; same planted attacks in every tier
Streaming parse and storage	**AttackStory**	Chunked parsers → Parquet; DuckDB with a hard memory limit
Detection	**AttackStory**	Sigma rules + sequence rules + rarity + known-bad patterns
ATT&CK and kill-chain labels	**AttackStory**	Deterministic mapping rules only
Linking into incidents and paths	**AttackStory**	Clusterer, tracer, look-ahead reachability
Delivery	**AttackStory**	API (`GET /incidents/{id}/path`), simple graph and timeline page, README, Docker Compose

What wins this PRD:

1. **Accuracy against the planted attacks:** over 90% of steps found, over 85% labelled correctly, every path in the right order, under 10% false incidents.
2. **Scale on a laptop:** the full tier in under an hour with under 8 GB of peak memory, then a bigger tier for the score multiplier if accuracy holds.

3. **Bonus features that are nearly free in our design:** risk score, plain-English report, alerts-merged count, Sigma rules from config.
4. **Trust:** narrative sentences cite their log lines, and log text cannot steer the AI.

PRD requirements and how we meet them

Every must-have maps to a component we already designed; FR-6 (look-ahead) and the performance targets are the two places the plan had to grow. Source: "PRD: Kill Chain and Attack Path Analysis", 30 October 2026.

Must have

ID	Requirement	Our component	Notes
FR-1	Read the full dataset without loading it into memory	Streaming ingest → Parquet; DuckDB `memory_limit`	See Performance design
FR-2	Detect suspicious events	Hunt engine	Add sequence rules, e.g. many failed logins then a success
FR-3	Map each detection to a technique and kill-chain stage	Mapping rules	Rules only; parent technique as the primary label
FR-4	Link related events into one incident by host, user, IP, time	Clusterer + linking rules	
FR-5	Show the path as a timeline and a host/user graph	Kill map, host-hop view, timeline	Simple page is enough
FR-6	Show what the attacker could reach next	Look-ahead engine	Built from logs alone; see Look-ahead
FR-7	Simple API or page, short README	FastAPI	`GET /incidents`, `GET /incidents/{id}/path`, `GET /incidents/{id}/report`

Bonus

ID	Bonus	Our plan
B-1	Risk score per incident, worst on top	Incident score from the hunt engine, presented as risk
B-2	Auto-written plain-English report	LLM narrator + citation checker, generated offline after processing
B-3	Raw alerts merged into each incident	Count of hunt hits per incident, shown on every incident
B-4	New detection rule via config, not code	Load Sigma and our YAML sequence rules from a `rules/` folder at start-up
B-5	Live mode	Stretch: tail new lines, update the affected incidents within seconds

Scoring targets

Measure	Good	Great	Our design lever
Detection (planted steps found)	> 70%	> 90%	Rules for the PRD's attack story + rarity + tracer pulling in un-flagged steps
Correct labels	> 60%	> 85%	Deterministic mapping; parent technique primary
Attack path (right steps, right order)	Half	All	Order by timestamp inside each incident
False alarms (incidents that are not attacks)	< 30%	< 10%	Kill-chain score threshold; report few, strong incidents
Processing time (full tier)	< 3 h	< 1 h	Parallel streaming parse, Parquet, columnar queries
Peak memory	< 16 GB	< 8 GB	DuckDB memory limit, chunked parsing, graph only over hits
Time to open one incident path	< 2 s	< 500 ms	Everything precomputed and saved at the end of processing

Tier choice

Tier	Size	Multiplier	Our plan
Core	15 GB	1.0×	Default target
Advanced	30 GB	1.3×	Switch if our synthetic 15 GB run finishes well under an hour and under 8 GB with accuracy intact
Extreme	50 GB	1.6×	Only with a multi-core cloud machine

The accuracy gate means a bigger tier only pays if accuracy stays above the bar, so decide after benchmarking, not before.

What goes in and what comes out

In: the organisers' log dataset (one tier). Out: a ranked list of incidents, each with labelled steps, an ordered path, a look-ahead, a risk score and a report, served by a small API. No alerts, no SIEM, no hints about where the attacks are.

Input: the competition dataset

- **Dev sample:** 100 MB, shared at kickoff, for building and profiling formats.
- **Full tier:** 15, 30 or 50 GB uncompressed, downloaded when the build starts; the same formats and planted attack sequences at every size.
- **Formats:** shared with the Log Parser problem and unknown until kickoff. The ingest layer must sniff formats and map them to one event schema within the first hours.
- **Environment in the story:** a company with a few hundred laptops, servers and users; an attacker guesses a password, gets onto a laptop, moves to a file server and copies data out.

Output: incidents, paths and reports


```
{
  "dataset": "core-15gb",
  "processed": {"bytes": 16106127360, "events": 98000000, "seconds": 2710,
"peak_memory_gb": 6.8},
  "incidents": [
    {
      "incident_id": 12,
      "risk": 0.93,
      "alerts_merged": 214,
      "steps": [
        {"order": 1, "time": "...", "host": "HR-07", "user": "asha",
"technique": "T1110", "sub_technique": "T1110.001", "stage": "Credential
Access", "evidence": ["E1041", "E1042"]},
        {"order": 2, "time": "...", "host": "HR-07", "user": "asha",
"technique": "T1078", "stage": "Initial Access", "evidence": ["E1043"]}
      ],
      "path": ["HR-07", "FS-02", "DC-01"],
      "look_ahead": [{"account": "svc_backup", "can_reach": ["BK-01", "FS-
03", "SQL-02"], "why": "admin logons seen on these hosts; credentials exposed
on FS-02"}],
      "report_url": "/incidents/12/report"
    }
  ]
}
```


API (FR-7)

Endpoint	Returns	Target latency
`GET /incidents`	Incidents ranked by risk, with alerts merged and stages covered	< 500 ms
`GET /incidents/{id}/path`	Ordered steps, host/user graph, timeline	< 500 ms
`GET /incidents/{id}/lookahead`	Accounts and hosts the attacker could reach next, with reasons	< 500 ms
`GET /incidents/{id}/report`	Plain-English report with citations (B-2)	< 500 ms
`GET /stats`	Processing time, peak memory, events, hits, incidents	< 500 ms

All responses are read from results computed during processing, so opening an incident never re-scans the logs.

Done means

- Over 90% of planted steps found and over 85% labelled with the right technique.
- Every planted attack rebuilt with the right steps in the right order.
- Under 10% of reported incidents are not real attacks.
- The chosen tier processed in under an hour with under 8 GB peak memory.
- Any incident path opens in under 500 ms.
- Judges can start everything from the README with Docker Compose.

Development data before the real dataset arrives

Build and benchmark against a synthetic generator that plants attacks and writes its own answer key; the VM lab becomes optional, kept only for a realistic demo video. The organisers' dataset is what gets scored, and its formats are unknown until kickoff.

Source	What it gives	When
Synthetic generator (ours)	Any size up to 15+ GB, planted attacks, exact ground truth, tunable noise	From Day 1; main source for accuracy and performance testing
OTRF Security-Datasets	Real Windows and Linux telemetry with recorded attacks	From Day 1; realism check for parsers and rules
Organisers' dev sample (100 MB)	The real formats and field names	At kickoff; parsers and rules adapted within hours
Dhan Securities lab (optional)	Real AD, Sysmon and Caldera attacks	Only if a person is free; for the demo video

Synthetic generator

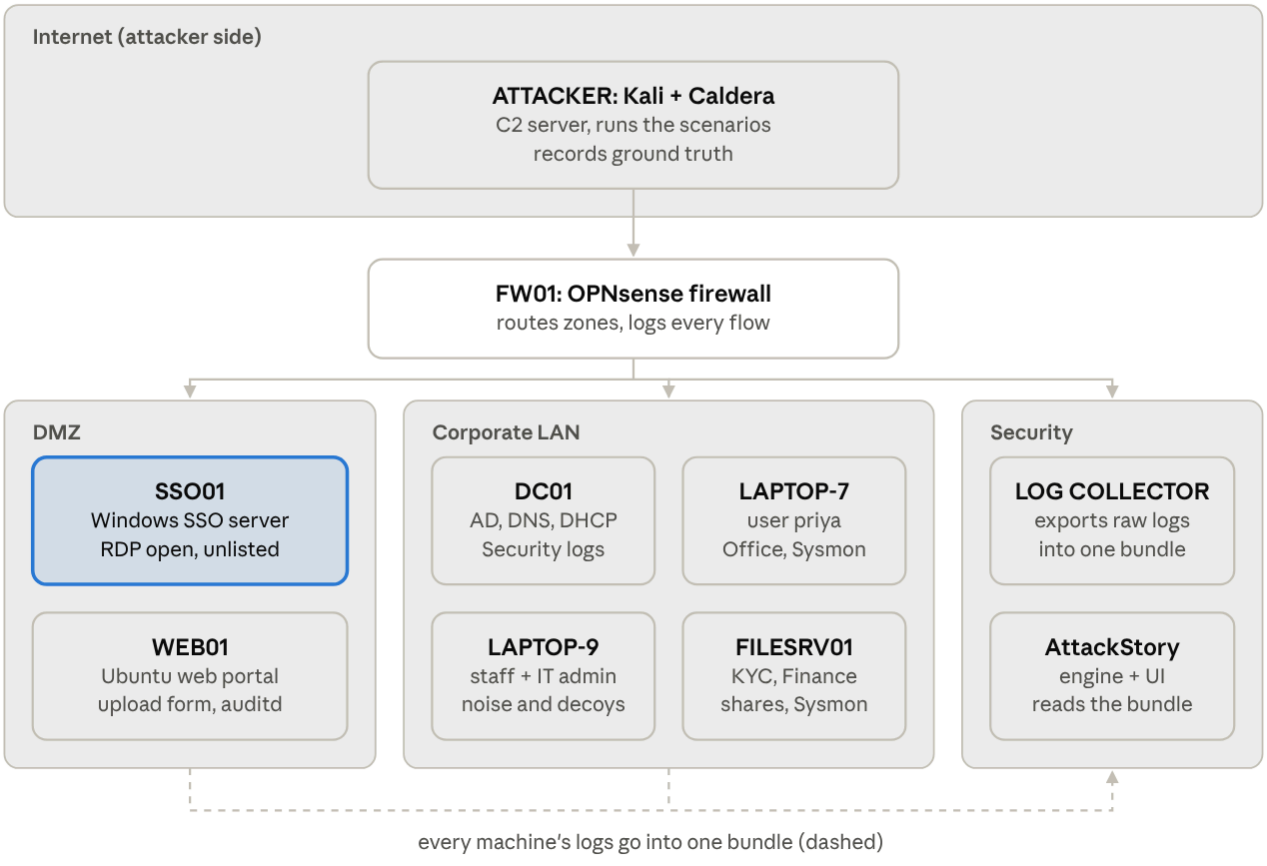
1. **World:** a few hundred users, laptops and servers (HR-07-style names), a domain controller, file servers, normal schedules, admins, service accounts.
2. **Normal activity:** logons, file access, process starts, DNS, web traffic, backups and admin work at realistic daily rhythms, including lookalikes such as legitimate PsExec and occasional mistyped passwords.

3. **Planted attacks:** the PRD's story and variants (password guessing → laptop → file server → domain controller → data out), each step written as log lines in the expected formats and recorded in `ground_truth.json` with technique, stage, host, user and time.
4. **Formats:** emit several common formats (Windows Security events as JSON, Linux auth/syslog, firewall CSV, web access logs) so the parsers are ready for whatever the dev sample shows; switch to the real formats at kickoff.
5. **Scale:** stream output to disk so a 15 GB set can be generated for speed and memory benchmarks.

Optional demo lab: Dhan Securities

A fictional Indian stockbroker on 9 virtual machines, built so a CDSL-style breach can be re-enacted for a demo video. Skip it if the team is short of time; nothing in scoring depends on it.

Dhan Securities lab: 9 machines, one unlisted internet-facing server



Dhan Securities lab · 9 VMs in 4 zones

SSO01 is the deliberate weak point: domain-joined, reachable from the internet over RDP, and missing from the critical-asset list, exactly the pattern SEBI found at CDSL (order coverage).

Machines

Host	OS	Role	Telemetry	RAM (GB, estimate)
ATTACKER	Kali Linux	Caldera server, C2, attack tools; keeps the ground-truth run log	none	4
FW01	OPNsense	Routes Internet, DMZ, Corp and Security zones	Firewall flow logs via syslog	1
SSO01	Windows Server 2019	Internet-facing, domain-joined, RDP open; the CDSL analogue	Sysmon, Security, PowerShell 4104	3
WEB01	Ubuntu 22.04	Customer portal with file upload; config holds a domain service account	auditd, web access log	2
DC01	Windows Server 2019	AD DS, DNS, DHCP	Security, Sysmon, DNS and DHCP logs	4
LAPTOP-7	Windows 10/11	User priya; Office with macros allowed	Sysmon, Security, PowerShell 4104	4
LAPTOP-9	Windows 10/11	User rahul + IT-admin activity (noise and decoys)	Sysmon, Security, PowerShell 4104	4
FILESRV01	Windows Server 2019	Shares: KYC, Finance, Test-Ransom	Sysmon, Security	3
WAZUH	Ubuntu	Log collector only (Wazuh archives or file export); its own alerts are kept aside as a baseline. AttackStory runs beside it	n/a	8

Total is roughly 33 GB of RAM. For reference, GOAD's MINILAB alone needs about 8–12 GB, GOAD-Light about 20 GB and full GOAD 24 GB or more (GOAD variants, OWASP VWAD listing). **Minimum build** if hardware is short: drop LAPTOP-9 and WEB01 (you lose

Scenario C and some noise), about 25 GB; or run the lab on cloud credits.

Deliberate weaknesses

Weakness	Where	Used in	Real-world mirror
Admin account password never expires, and is weak	AD (svc_admin)	Scenario A	SEBI's CDSL findings
No MFA on privileged accounts	AD	A	SEBI's CDSL findings
RDP reachable from the internet	SSO01	A	SEBI's CDSL findings
Server missing from the critical-asset list	SSO01	A (report finding)	CDSL's ADFS server
Office macros allowed; user is local admin	LAPTOP-7	B	Common phishing path
Flat network: SMB open from every zone to FILESRV01	FW01	A, B, C	Common in SMEs
Outbound 443 allowed to anywhere	FW01	B	Exfiltration path
File upload without validation	WEB01	C	Web shell entry
Plaintext service-account password in app config	WEB01	C	Credentials in files

SSO01 still sends logs in the lab so the attack can be reconstructed; its "unlisted" status appears in the incident report as a governance finding. **Safety:** the lab is isolated, has no route to the real internet, and the fake ransomware only renames dummy files in the Test-Ransom share.

Telemetry and the log bundle (optional lab only)

Only needed if the team builds the demo lab: collect what the three scenarios need, export it as one bundle per run, and check every technique's events land in the bundle. For the competition itself, the organisers' dataset replaces all of this.

What each source must give us

Source	Hosts	Events and fields we rely on	Used for
Sysmon (community config: SwiftOnSecurity or Olaf Hartong's sysmon-modular)	All Windows	1 process create (command line, parent, ProcessGuid, hashes) · 3 network connection · 8 remote thread · 10 process access (LSASS) · 11 file create · 13 registry set · 22 DNS query	Process trees, credential theft, persistence, C2
Windows Security (advanced audit policy)	All Windows	4624 logon (type 3 network, 10 RDP; source IP; LogonId) · 4625 failed logon · 4648 explicit credentials · 4672 admin logon · 4688 process create with command line · 4697 and 7045 service install · 4698 scheduled task · 4720 user created · 4728 and 4732 group add · 4769 Kerberos ticket · 1102 log cleared	Cross-host links, accounts, persistence
PowerShell Operational	All Windows	4104 script block (decoded script text)	What encoded PowerShell actually ran
DNS and DHCP server logs	DC01	Lease: IP → hostname at a time · queries	Entity resolution (which machine had an IP)
auditd	WEB01	execve with parent · connect · writes under the web root and /etc/cron.d · reads of the app config	Web shell, reverse shell, persistence

Source	Hosts	Events and fields we rely on	Used for
Web server access log	WEB01	Method, path, status, client IP, user agent	Upload of the web shell
Firewall filter log	FW01	Source, destination, port, action, bytes	Inbound RDP, exfiltration volume
Zeek (optional)	Mirror port on FW01	conn, dns, http, ssl logs	Richer network evidence

Building the log bundle after each run

1. Sync every machine to one NTP source before the run. Clock skew of even a minute breaks cross-host ordering.
2. After the run, export Windows channels from every Windows host: `wevtutil epl Security`, `Microsoft-Windows-Sysmon/Operational`, `Microsoft-Windows-PowerShell/Operational` and `System`, each to its own `.evtx` file.
3. Copy Linux logs from WEB01: `/var/log/audit/audit.log`, `auth.log`, and the web server's access log.
4. Copy FW01's firewall syslog and DC01's DHCP and DNS logs.
5. Write `manifest.json` (host, OS, time zone per file) and zip it all as `dhan-run-NN.zip`. Keep the Caldera run log **outside** the bundle; it is the answer key.
6. Optional shortcut: run Wazuh with archive logging on and export its archive per host. Wazuh's own alerts are saved separately as the "SIEM alerts" baseline, never given to AttackStory.

This is the same thing the organisers are likely to hand over: files, not a live system. Practise on bundles, not on a dashboard.

Validate before building scenarios

Run each technique once with Atomic Red Team, export a bundle, and tick it off only when the expected event is in it. A public Wazuh + Sysmon lab did exactly this and found Sysmon silently dropping events, custom rules losing to built-in ones, and Windows Defender blocking a technique before Sysmon saw it (SIEM-Lab write-up). Budget time for the same surprises.

- [ ] LSASS access (T1003.001) produces Sysmon 10 in the bundle
- [ ] PsExec lateral movement produces 4624 type 3 and 7045 in the bundle
- [ ] Encoded PowerShell produces 4104 with decoded text

- [ ] RDP brute force produces a 4625 burst in the bundle
- [ ] Web shell write produces an auditd event on WEB01
- [ ] Firewall flow logs show bytes for outbound 443
- [ ] DHCP lease log maps LAPTOP-7 to its IP

Attack scenarios

The PRD's own story tells us which attacks are planted, so detection rules for these come first; the three lab scenarios below are richer variants for the generator and the optional demo.

Attack steps the PRD describes, and the rules to build first

Step in the PRD story	Rule idea	Technique	Stage
Many failed logins on one account, then a success	Sequence rule: ⩾ N failures then success within a window, same account or source	T1110 Brute Force	Credential Access
Logon with the guessed account	Success right after the brute-force sequence, or from a new source	T1078 Valid Accounts	Initial Access
Discovery on the laptop	Enumeration commands, many lookups in a short burst	T1087, T1018, T1083	Discovery
Move laptop → file server → domain controller	First-time network or remote logons between hosts by the same account	T1021 Remote Services	Lateral Movement
New admin account	Account created then added to a privileged group	T1136, T1098	Persistence
Copy data	Large or unusual file reads, archive creation on the file server	T1005, T1560	Collection
Big upload to an outside IP	Outbound volume from an internal host to a new external IP far above its baseline	T1041, T1048	Exfiltration

Threshold values (how many failures, how many bytes) are set from the dev sample's baselines, not guessed.

Scenario A: "The unlisted gateway" (CDSL-style)

Based on SEBI's findings at CDSL: an internet-facing server outside the critical-asset list, a never-expiring admin password, no MFA, an untested internet-reachable RDP port, and settlement disrupted for 46 hours (source).

#	Time	Action	Host	ATT&CK	Evidence we expect
1	T+0	Password guessing against RDP from the internet	SSO01	T1110.001	4625 burst from attacker IP; firewall allows 3389
2	T+5	RDP logon as svc_admin (never-expiring password)	SSO01	T1078.002	4624 type 10 + 4672
3	T+8	Domain discovery (`net group` , `nltest`)	SSO01	T1087.002, T1482	Sysmon 1 / 4688 command lines
4	T+12	Dump LSASS memory	SSO01	T1003.001	Sysmon 10 (target lsass), Sysmon 11 dump file
5	T+18	PsExec to the domain controller with stolen admin credentials	DC01	T1021.002, T1569.002	4624 type 3 from SSO01's IP; 7045 PSEXESVC
6	T+25	Create a backdoor domain admin	DC01	T1136.002, T1098	4720, 4728
7	T+30	Scheduled task pushes the fake-ransomware script	FILESRV01	T1053.005	4698; Sysmon 1
8	T+32	Delete shadow copies	FILESRV01	T1490	Sysmon 1 `vssadmin delete shadows`
9	T+33	"Encrypt" dummy files (rename to .locked)	FILESRV01	T1486	Burst of Sysmon 11 in Test-Ransom

Scenario B: "Phish to KYC exfiltration" (with injection test)

#	Time	Action	Host	ATT&CK	Evidence we expect
1	T+0	priya opens a macro document (dropped to simulate email)	LAPTOP-7	T1566.001, T1204.002	Sysmon 1: WINWORD.exe → powershell.exe -enc
2	T+1	PowerShell downloads a stager; **command line carries a planted note telling AI analysts the script is approved**	LAPTOP-7	T1059.001, T1105	4104 script text; Sysmon 3 + 22 to C2

#	Time	Action	Host	ATT&CK	Evidence we expect
3	T+3	Run-key persistence	LAPTOP-7	T1547.001	Sysmon 13
4	T+5	LSASS access	LAPTOP-7	T1003.001	Sysmon 10
5	T+11	Network logon to the file server	FILESRV01	T1021.002	4624 type 3, source = LAPTOP-7's IP
6	T+13	Compress the KYC share	FILESRV01	T1560.001	Sysmon 1 (archiver) + 11 (archive file)
7	T+20	Send the archive out over HTTPS	FILESRV01	T1041	Firewall bytes out; Sysmon 3

Scenario C: "Web shell to Windows share" (Linux entry)

#	Time	Action	Host	ATT&CK	Evidence we expect
1	T+0	Upload a PHP web shell through the unvalidated form	WEB01	T1190, T1505.003	Access log POST; auditd write in the upload folder
2	T+2	Run commands through the shell	WEB01	T1059.004	auditd execve, parent = web server process
3	T+4	Reverse shell to the attacker	WEB01	T1059.004, T1071.001	auditd connect; firewall outbound
4	T+6	Read service-account password from app config	WEB01	T1552.001	auditd read of the config file
5	T+9	Log into FILESRV01 over SMB with that account	FILESRV01	T1078.002, T1021.002	4624 type 3, source = WEB01's IP
6	T+12	Cron persistence	WEB01	T1053.003	auditd write in /etc/cron.d

Normal activity and decoys (what makes it hard)

A small script on every host simulates staff all day: logons, file opens, Office, web browsing, scheduled backups. Two **decoys** test precision:

- **IT admin PsExec:** the account it_admin runs PsExec from LAPTOP-9 to FILESRV01 for patching at T−20 and T+15 minutes. Same tool as the attack, but no link to it. It must stay out of the story.
- **Legit PowerShell:** a signed inventory script runs hourly on every laptop. Same binary, different parent and no credential access.

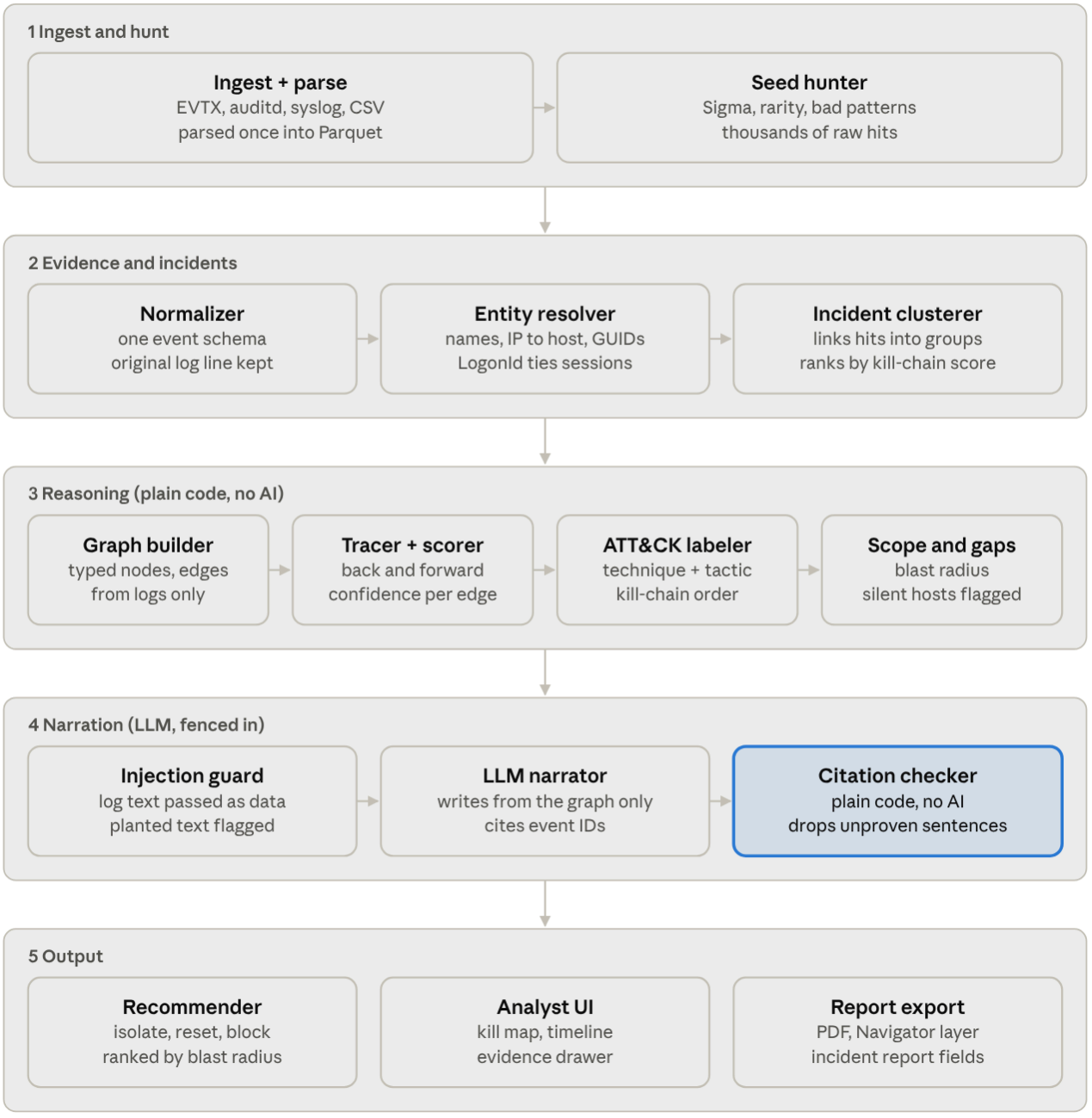
Ground truth

- Steps Caldera can run are built as one Caldera adversary profile per scenario; its operation report records every step with timestamps, host and ATT&CK technique (Caldera operation results).
- Steps from outside (RDP guessing, web upload) are scripted; the script appends to the same run log: step, technique, host, start, end.
- Run each scenario at least 3 times with different timing and noise seeds, so the engine is not tuned to one recording.

Engine architecture

Everything that decides what happened is plain code; the LLM only turns the finished graph into sentences, and a plain-code checker has the last word. This follows the 2026 CyberClear finding that evidence accumulation and step validation improve attack-chain reconstruction more than model choice (CyberClear).

The graph is built from logs before the LLM writes a word



AttackStory engine · 5 layers, 15 components

Hunt engine: finding the alerts ourselves

With no alerts given, the hunt engine creates its own seeds, then groups and ranks them so that only a handful of incidents reach the expensive tracer. Real attacks move through several tactics across linked machines; noise is usually one isolated tactic. Ranking on that difference is the core trick.

1. **Ingest once, at scale.** Detect each file's type; parse Windows logs with Hayabusa (JSON timeline output) or `evtx_dump`, both fast native tools; parse auditd, syslog, web and firewall logs into the same event schema. Write Parquet partitioned by host and

day, in parallel per file. DuckDB queries the Parquet directly, so gigabytes never have to fit in memory.

2. **Generate seeds three ways.**

Seed source	Examples	What it produces
Sigma rules	Hayabusa over EVTX; Zircolite or pySigma-to-SQL over JSON and auditd	Hits with ATT&CK tags and severity
Rarity	First-ever parent → child pair (WINWORD → powershell); first logon from host A to B; new external IP receiving large data; burst of 4625; new service or task	Anomaly hits with a rarity score
Known-bad patterns	The mapping rules in the ATT&CK section, run as queries: LSASS access, encoded PowerShell, `vssadmin delete shadows`, web server spawning a shell	High-precision hits

3. **Cluster.** Link hits with the same rules the graph uses (process lineage, LogonId, source IP → host, same account within 60 minutes). Each connected group is a candidate incident.
4. **Rank.** Score each cluster on tactic coverage, kill-chain order, hosts linked, worst severity and rarity:

```
S = 0.35\,\frac{T}{T_{max}} + 0.20\,O + 0.15\,\frac{\min(H,3)}{3} + 0.20\,V + 0.10\,R
```

T = distinct ATT&CK tactics in the cluster; O = share of step pairs in kill-chain order; H = hosts linked; V = highest Sigma severity scaled 0–1; R = mean rarity scaled 0–1. Weights are starting values to tune on the lab runs.

5. **Decide.** S ≥ 0.7 is reported as a likely attack; 0.4 to 0.7 as "review"; below 0.4 is excluded but counted and browsable. A bundle with no cluster above 0.7 returns "no confident attack found".
6. **Investigate.** Each reported cluster goes to the tracer below. Its highest-severity hit becomes the seed, and tracing pulls in the un-flagged events around it (the normal-looking logon, the file write) from the Parquet store.

Worked example (illustrative numbers): 8 GB from 7 hosts → about 4,200 Sigma hits plus 35 rarity hits → 61 clusters. 58 are single-tactic noise (an admin's PowerShell, a backup job) scoring under 0.4. One spans six tactics across three hosts in order (Word → PowerShell → LSASS → SMB logon → archive → 2 GB out) and scores 0.93. Two score in the review band.

Scale rules: parse once and query many times; filter every query by host and time; trace only the top clusters; the LLM never sees raw logs, only the final pruned story of tens of events.

Performance design (FR-1, time and memory targets)

Parse once in parallel into compact columns, run detection as columnar queries, and precompute every answer, so 15 GB finishes inside an hour under 8 GB. Throughput needed: about 4.3 MB/s for 15 GB in an hour, 8.5 MB/s for 30 GB and 14.2 MB/s for 50 GB.

1. **Stream:** read each file in line batches (e.g. 100,000 lines); never read a whole file.
2. **Parallelise:** one worker per CPU core; large files are split into byte ranges at line boundaries; each worker writes Arrow batches to Parquet, partitioned by day and host.
3. **Compact:** keep only needed fields; dictionary-encode hosts, users and event types; store timestamps as integers; compress with zstd.
4. **Cap memory:** DuckDB `SET memory_limit='6GB'` with spill-to-disk; the graph holds only hunt hits plus traced context (thousands of events, not millions).
5. **Detect in columns:** rules compile to SQL over Parquet; sequence rules such as failures-then-success use window functions, one pass per rule family.
6. **Precompute:** after clustering, write each incident's path, look-ahead, risk and report to a results store; the API only reads it.
7. **Measure every run:** wall time and peak memory (RSS) logged per stage and exposed on `/stats`.

Stage	Budget for 15 GB (planning target)
Parse and write Parquet	25 min
Detection queries	10 min
Clustering and tracing	5 min
Look-ahead and reports	5 min
Slack	15 min

If Python parsing misses its budget on the synthetic 15 GB run, move the hot parser to Go or Rust, or let DuckDB read structured formats (CSV, JSON lines) natively.

Graph model

Node type	Unique key	Example
User	Account, lower-cased, domain stripped to one form	dhan\priya
Host	Hostname; IPs resolved to a host via the DHCP lease at that moment	LAPTOP-7
Process	Sysmon ProcessGuid + host	powershell.exe {45cd…}
Logon session	LogonId + host	0x3E7A2 on FILESRV01
File or share	Full path	\FILESRV01\KYC\clients.xlsx
Registry key	Full path	HKCU\…\Run\updater
Network endpoint	IP:port	203.0.113.50:443
Account object	SID	backdoor admin created on DC01

Linking rules and confidence

Link	How we know	Starting weight
Spawned (parent → child)	Sysmon 1 ParentProcessGuid = ProcessGuid	1.0
Accessed (process → process)	Sysmon 10 SourceProcessGuid	0.95
Session ran (logon → process)	Same LogonId on the same host (4624 → 4688)	0.9
Wrote or read (process → file)	Sysmon 11, auditd	0.9
Connected (process → endpoint)	Sysmon 3, auditd connect	0.9
Logged on (host A → host B as user)	4624 type 3/10 on B whose source IP was A's at that time	0.8
Sent bytes (host → external)	Firewall flow on host IP + time	0.7

Link	How we know	Starting weight
Reused credential	Account touched by a credential-theft step logs on elsewhere within 60 min	0.6
Time proximity only	Same host, within 5 min, no other tie	0.3

When several rules support the same edge, combine them with noisy-OR, so independent weak clues add up without exceeding 1:

```
c = 1 - \prod_{i} (1 - w_i)
```

Edges with c ⩾ 0.7 enter the story as solid; 0.4 to 0.7 appear dashed as "possible"; below 0.4 they are dropped. All weights are starting values to tune on the lab runs.

Tracing algorithm

1. **Seed:** the highest-severity hunt hit in the incident cluster, plus the cluster's other hits.
2. **Backward:** follow incoming spawned, session and logon edges to the earliest ancestor that is an entry point (internet logon, web request, user-opened document) or a host boundary. Stop at boot-time system roots unless they link to attacker activity.
3. **Forward:** walk outgoing edges in time order from the seed and every ancestor, within +6 hours by default, through edges with c ⩾ 0.4.
4. **Credential watch:** once a credential-theft step is found, any later logon by that account on any host becomes a candidate edge.
5. **Prune:** drop nodes with no path to the seed; collapse repeated benign children (for example conhost.exe).
6. **Excluded list:** events that look like attack techniques but fail linkage (the IT admin's PsExec) are kept in a "considered and excluded" list with the reason, shown in the UI.

Gaps and blast radius

- **Gaps:** each host has an expected event rate per source. A window with no events from a host inside the story becomes a gap; any edge crossing it is marked inferred, never observed.
- **Blast radius, confirmed:** users, hosts, shares, files and external IPs touched by story edges.
- **Blast radius, potential:** hosts reachable next, computed by the look-ahead engine below from privileges and logons seen in the logs.

Look-ahead: what the attacker could reach next (FR-6)

Built from the logs alone: start from the accounts and hosts the attacker controls, add accounts whose credentials were exposed on those hosts, then list every host where any of them has shown admin access. No Active Directory query is needed.

1. **Compromised set:** accounts the attacker used, guessed or created, and hosts the incident touched.
2. **Credential exposure:** accounts that logged on interactively to a compromised host during or before the attack; their credentials may be stealable there.
3. **Privilege map (built during ingest for the whole dataset):** for each account, hosts where it logged on with admin privileges (Windows special-privilege logons, sudo), and group additions that granted admin rights.
4. **Network reach:** hosts the compromised hosts already connected to successfully over admin protocols (SMB, RDP, SSH, WinRM).
5. **Reach list:** hosts where any at-risk account has admin access, plus network-reachable admin targets, minus hosts already touched; ranked by criticality (domain controllers, file servers, databases, backups first).

Worked example: the attacker controls `asha` on HR-07 and reached FS-02. While on FS-02, `svc_backup` had an interactive logon there, so its credentials are exposed. The privilege map shows `svc_backup` with admin logons on BK-01, FS-03 and SQL-02. Output: "`svc_backup`'s credentials were exposed on FS-02; this account has admin rights on 3 more servers: BK-01, FS-03, SQL-02."

Rights are labelled "observed admin access" because logs show what accounts did, not everything they are allowed to do.

Narrator contract

The LLM receives only the pruned story: steps with event IDs, normalized fields and ATT&CK labels, in a JSON data block. It returns JSON sentences, each with `text`, `citations` and `kind` (observed or inferred).

Citation checker (plain code):

1. Every sentence has at least one citation.
2. Every cited event exists in the story.
3. Every host, user, IP and file named in the sentence appears in a cited event.
4. Every ATT&CK ID named matches a cited step's label.
5. Every time named matches a cited event within one minute.

A failing sentence is dropped. If more than 20% are dropped, regenerate once; if it fails again, fall back to a template narrative built from the steps without an LLM.

Generated offline (B-2): reports are written once, after processing, and saved; the 500 ms API path never waits on an LLM.

Injection guard

- Log fields (command lines, file names, script text, user agents) are only ever passed as JSON values inside a data block, never spliced into instructions.
- A detector flags instruction-like text in those fields ("ignore previous", "classify as benign", "note to AI analyst"). The event is tagged and shown to the analyst as an indicator of attacker intent.
- The LLM cannot change the graph, the verdict or the labels; it only narrates, and the checker enforces that.

MITRE ATT&CK and kill-chain mapping

Label each step in three passes, from most to least trusted, and show which pass produced each label. Ordering then comes from the graph and the clock, not from the label alone.

Three labelling passes

1. **Sigma tag:** the Sigma rules our hunt engine runs attach technique IDs to their hits. Trust these first.
2. **Our mapping rules:** deterministic patterns for events no Sigma rule tagged (table below). Most story events come from here.
3. **LLM suggestion:** only for a story step that is still unlabelled. Shown with a "suggested" badge and never used for scoring unless an analyst confirms it.

Example mapping rules

Evidence pattern	Technique	Tactic
Office process spawns a script host (powershell, cmd, wscript)	T1204.002 User Execution: Malicious File	Execution
powershell.exe with `-enc` or a download cradle in 4104	T1059.001 PowerShell	Execution
Burst of 4625 from one source, then 4624 for the same account	T1110 Brute Force	Credential Access
Sysmon 10 with target lsass.exe from a non-system process	T1003.001 LSASS Memory	Credential Access

Evidence pattern	Technique	Tactic
Write to a Run key	T1547.001 Registry Run Keys	Persistence
4624 type 3 + 7045 PSEXESVC on the target	T1021.002 SMB/Admin Shares + T1569.002 Service Execution	Lateral Movement, Execution
4698 scheduled task created remotely	T1053.005 Scheduled Task	Execution, Persistence
4720 + 4728 (new user added to a privileged group)	T1136.002 Domain Account + T1098	Persistence
`net group`, `nltest /domain_trusts`	T1087.002, T1482	Discovery
Archiver run on a share, archive file written	T1560.001 Archive via Utility	Collection
Large outbound flow from a server to a new external IP	T1041 Exfiltration Over C2 Channel	Exfiltration
`vssadmin delete shadows`	T1490 Inhibit System Recovery	Impact
Mass renames in a share	T1486 Data Encrypted for Impact	Impact
auditd: web server process spawns a shell	T1505.003 Web Shell	Persistence
Read of an app config with credentials	T1552.001 Credentials in Files	Credential Access

Output convention for scoring: each step carries the parent technique (T1110) as its primary label, the sub-technique (T1110.001) in a separate field, and exactly one kill-chain stage. LLM-suggested labels never appear in the scored output.

Technique names and IDs are loaded from MITRE's ATT&CK STIX data at build time, so labels always match the current matrix.

Ordering into a kill chain

Use ATT&CK's tactic order as the lanes: Reconnaissance, Resource Development, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration, Impact.

- One technique can sit under several tactics (Valid Accounts is Initial Access, Persistence, Privilege Escalation and Defense Evasion). Pick the tactic from the step's position in the graph: the first logon from the internet is Initial Access; the same

account used later on another host is Lateral Movement.

- Within a lane, sort by time. Across lanes, the timeline is the truth: if the clock says credential access happened before execution on another host, show it that way and flag it rather than forcing textbook order.

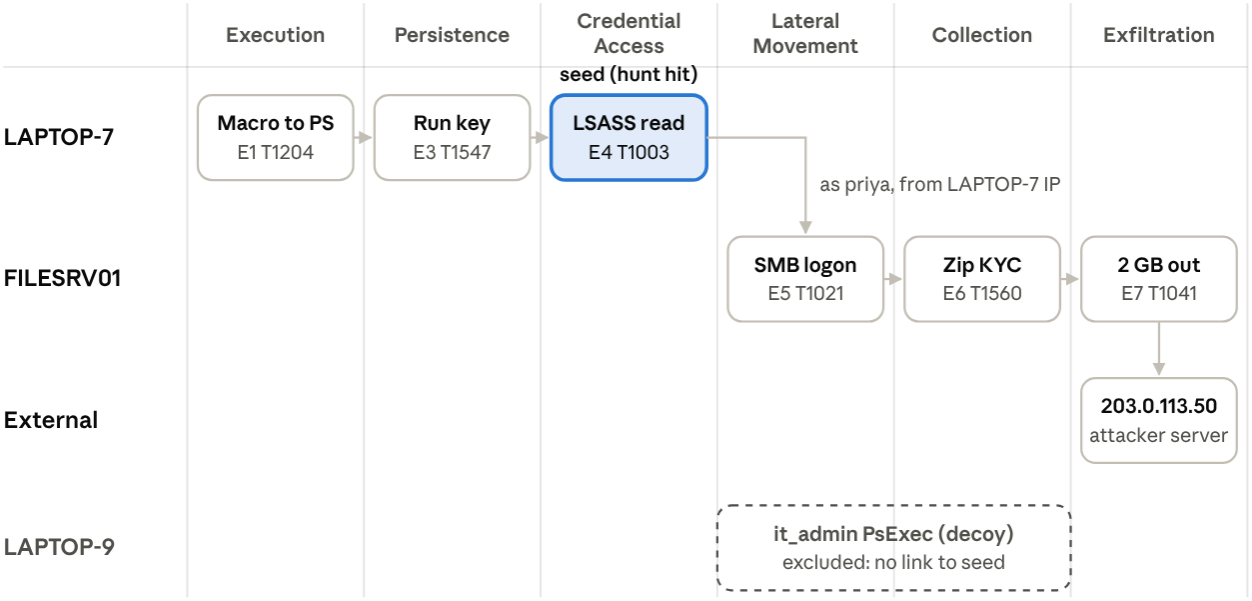
What the analyst gets

- A per-step label with its source (Sigma tag, mapping rule, or suggested).
- An ATT&CK matrix heatmap of the incident.
- An export as an ATT&CK Navigator layer, so analysts can open the incident in MITRE's own tool and compare it to threat-group profiles.

Kill map and analyst UI

The kill map puts tactics left to right and hosts top to bottom, so the attacker's path reads like a sentence and every hop between machines is visible. This mock-up shows Scenario B. The PRD does not require a polished UI, so this is one simple page over the API; accuracy and speed come first.

Kill map: one LSASS hunt hit grows into the full story across three hosts



Blue = seed, the top hunt hit. Arrows = links proven by logs. Dashed box = activity the engine considered and excluded.

kill map mock-up · Scenario B

The IT admin's PsExec uses the same tool as the attack but has no proven link to the seed, so it is shown as excluded instead of silently dropped. That one box answers the judge's question "how do you avoid false positives?"

Features

Feature	What the analyst sees	Why it matters
Kill map	Tactic lanes × host rows; nodes = steps with event ID and technique; seed highlighted	The whole attack at one glance
Host-hop view	Machines only, with the account used on each hop	Shows lateral movement and which credentials to reset
Timeline	Steps in time order with tactic bands and gaps between steps	Dwell time and speed of the attack
Evidence drawer	Click any node, edge or sentence: raw log lines with matched fields highlighted	Every claim is checkable in one click
Narrative panel	Story sentences with citation chips; toggle "observed only"	Readable story without trusting the AI blindly
Confidence slider	Hide or show dashed "possible" links by threshold	Analyst controls how speculative the view is
Excluded list	Lookalike activity left out, with the reason	Proves precision; builds trust
Gaps panel	Hosts or windows with no logs, and which links cross them	Honest about blind spots
Blast radius	Confirmed users, hosts, shares, external IPs; "could reach" list from admin rights	Tells responders what to contain
ATT&CK heatmap + Navigator export	Matrix of techniques in this incident; one-click layer file	Speaks the SOC's standard language
Replay	Scrub a time slider and watch the graph grow step by step	Best live-demo moment
Injection banner	"Instruction-like text found in E2's command line" with the text quoted	Shows the AI cannot be talked out of a verdict
Response checklist	Isolate, reset, block, hunt; each with owner and status	Turns the story into action

Feature	What the analyst sees	Why it matters
Incident report	Executive summary, technical narrative, ATT&CK table, blast radius, evidence appendix, recommended actions	Ready for management, regulator and insurer
Analyst feedback (stretch)	Confirm or reject an edge	Tunes link weights over time

Hunt overview (the first screen in logs-only mode): files and events processed, raw hits, clusters, and the ranked incidents. Each cluster shows its score breakdown (tactics, order, hosts, severity, rarity), so anyone can see why the attack ranked first and the noise did not.

Screen layout

- **Left:** the ranked incident list from the hunt, each with its score.
- **Centre:** kill map (default) with tabs for host-hop view and timeline.
- **Right:** narrative on top, evidence drawer below, opening on click.
- **Top bar:** blast-radius counters, injection banner when triggered, export button.

Evaluation

Measure exactly what the PRD measures (steps found, labels right, paths in order, false incidents, time, memory, latency) on every run, against ground truth we control, so the dataset at kickoff holds no surprises. No target numbers are claimed until runs produce them.

Ground truth

1. **Synthetic generator:** writes `ground_truth.json` for every planted step (technique, stage, host, user, time), so scoring is automatic at any size.
2. **OTRF Security-Datasets:** independent recordings mapped to ATT&CK (OTRF via MSTICPy).
3. **Dev sample:** after kickoff, label its attack steps by hand if no answer key is shared, and re-run the scorer.
4. **Optional lab:** Caldera operation reports give ground truth for the demo scenarios.

Scoring matches the PRD's definitions: a planted step counts as found when an incident contains an event from that step; a label counts as right when the parent technique matches; a path counts when every step is present in the right order; a false incident is

one with no planted step in it.

Metrics

Metric	Definition
Event precision	Story events that are truly malicious ÷ all story events
Event recall	Malicious events in the story ÷ all malicious events
Step recall	Attack steps with at least one story event ÷ all attack steps
Entry point	Did the story start at the true first step? (yes/no)
Hop recall	Lateral hops found ÷ true lateral hops
Technique accuracy	Steps with the exact technique ID; also reported at tactic level
Order accuracy	Share of step pairs shown in the true order
Decoy inclusions	Count of benign lookalike events pulled into the story (target: 0)
Grounding rate	Narrator sentences passing the checker before any are dropped
Injection resistance	Planted instruction variants that changed the story or verdict (target: 0)
Time to story	Minutes from log upload to ranked, finished incidents

Hunt-mode metrics (because no alert tells us where to look):

Metric	Definition
Incident recall	Attacks in the bundle that appear as incidents scoring ≥ 0.7 ÷ attacks in the bundle
True-attack rank	Position of the real attack in the ranked list (target: 1)
False incidents	Incidents scoring ≥ 0.7 that contain no malicious events (target: 0)

Metric	Definition
Clean-bundle result	On a run with no attack, does anything score ⩾ 0.7? (target: no)
Hit reduction	Raw hunt hits ÷ events shown to the analyst

Baselines

Baseline	What it is
SIEM alerts only	Wazuh's alerts on the same run, as a list: what a SOC with a SIEM would see
Hayabusa timeline	Sigma-tagged timeline of the same logs (Hayabusa)
LLM-only	The same LLM given Hayabusa's top hits (raw gigabytes do not fit) and asked to reconstruct the attack
AttackStory	The full engine

Ablations

Re-run AttackStory with one part removed each time: no entity resolution, no confidence scoring (all links equal), no citation checker, no injection guard. Each removal should hurt a specific metric; that proves every component earns its place.

Run plan

- Synthetic sets at 100 MB, 1 GB and 15 GB, three random seeds each, plus a clean set with no attack; OTRF recordings; the dev sample once it arrives.
- Performance runs on the synthetic 15 GB set (and 30 GB before choosing a bigger tier), logging time and peak memory per stage.
- One results table for the slide, with the PRD's columns: detection, correct labels, paths rebuilt, false alarms, processing time, peak memory, time to open an incident.

Tech stack and repo

Python + DuckDB for the engine, parallel parsers into Parquet, Sigma and sequence rules from config, FastAPI over precomputed results, a synthetic generator for testing; everything runs locally on one laptop.

Layer	Choice	Why
Development data	Our synthetic generator (Python, streaming writer); OTRF datasets	Ground truth at any size
Parsing	Python multiprocessing + PyArrow; Go or Rust for the hot parser if needed; Hayabusa only if EVTX appears	Hit the throughput budget
Store	Parquet partitioned by day and host; DuckDB with `memory_limit`	Gigabytes under 8 GB of memory
Detection	Sigma rules (pySigma to SQL) + our YAML sequence rules, loaded from `rules/`	B-4: new rules without code changes
Graph	NetworkX over hits and traced context	Small, fast graphs
ATT&CK data	mitreattack-python (STIX)	Current technique names and tactics
Engine API	Python 3.11 + FastAPI + Pydantic, reading a precomputed results store	Incident paths in under 500 ms
LLM	Any API with JSON output; local model as fallback; run offline	Report narration only (B-2)
UI	One page: React or plain HTML + Cytoscape.js	Graph and timeline (FR-5)
Measurement	psutil for peak memory; per-stage timers	Proves time and memory targets
Packaging	Docker Compose + README	Judges start it themselves
Optional lab	GOAD MINILAB or Ludus, Sysmon, Caldera	Demo video only

```
attackstory/
  generator/       synthetic world, normal activity, planted attacks,
ground_truth.json
  engine/
    ingest/        format sniffing, parallel parsers, to_parquet.py
    store/         duckdb views over parquet, schema.py
    hunt/          sigma runner, sequence rules, rarity scorers, known-bad
queries
    cluster/       hit linking, incident risk scoring and ranking
    entities/      user, host, process, session resolution
```

```
    graph/          builder, link rules, confidence, tracer
    attack/         mapping rules, ATT&CK loader, kill-chain stages
    lookahead/      privilege map, credential exposure, reach list
    narrate/        offline report writer, injection guard, citation checker,
template fallback
    results/        precomputed incidents, paths, look-aheads, reports
    api/            FastAPI: /incidents, /incidents/{id}/path, /lookahead,
/report, /stats
  rules/            sigma/ and sequences/ (YAML), editable without code
changes
  ui/               single page: incident list, path graph, timeline
  eval/             scorer for every PRD metric, benchmark runner
  lab/              optional demo lab
  docker-compose.yml
  README.md
```

Agree `engine/store/schema.py` (the normalized event) and the incident JSON on Day 1; every other module codes against those two contracts.

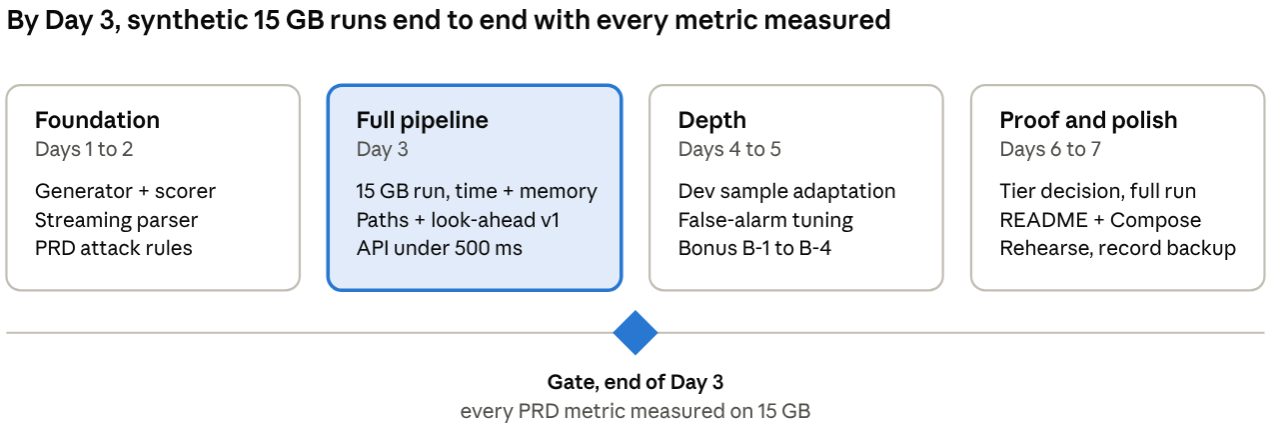
Team and 7-day plan

Five owners, two shared contracts (event schema and incident JSON), and one gate: by the end of Day 3 the synthetic 15 GB set must run end to end with every PRD metric measured.

Person	Owns	First deliverable
P1 Ingest and performance	Format sniffing, parallel streaming parsers, Parquet, DuckDB limits, time/memory benchmarks, tier decision	1 GB synthetic set parsed with time and peak memory logged
P2 Detection and labels	Sigma and sequence rules, known-bad and rarity queries, ATT&CK and stage mapping, rules from config (B-4)	Brute-force-then-success and big-upload rules firing on synthetic data
P3 Correlation and paths	Entity resolution, clustering, tracer, path ordering, look-ahead (FR-6), risk score (B-1)	One planted attack rebuilt as an ordered path

Person	Owns	First deliverable
P4 Evaluation and data (Keshav)	Synthetic generator with ground truth, scorer for every PRD metric, false-alarm tuning, offline report writer (B-2)	Generator + scorer producing the PRD results table
P5 API and delivery	FastAPI endpoints over precomputed results, single-page graph and timeline, README, Docker Compose	`GET /incidents/{id}/path` under 500 ms

At kickoff: everyone spends the first two hours on the 100 MB dev sample: P1 maps its formats, P2 checks which rules fire, P3 checks the joins (host, user, IP names), P4 hand-labels the attack steps it can see, P5 points the API at the new results.



7-day plan · 4 phases, 1 gate

Everyone develops against the synthetic generator and OTRF from Day 1, so nobody waits for the real dataset. If the gate slips, cut live mode (B-5), the optional lab and UI polish first; protect accuracy, the time and memory targets, the look-ahead and the results table, because those are what the judges score.

Pitch and judge Q&A

Lead with the scorecard against their planted attacks, prove speed and memory, then open one incident and walk its path and look-ahead. Judges score accuracy and scale first; the story is how they remember you.

Demo script

1. **0:00 Hook.** "An attacker guessed a password, reached a laptop, a file server and the domain controller, and took data out. Each step is one line among millions. We find all of them in under an hour."
2. **0:30 Scale.** `/stats`: tier processed, time, peak memory, events, raw hits, incidents.
3. **1:00 Scorecard.** Detection, correct labels, paths rebuilt, false alarms against the ground truth, next to the Good and Great targets.
4. **1:40 Incident list.** Ranked by risk (B-1), each showing how many raw alerts it merged (B-3).
5. **2:10 One incident.** Opens in under 500 ms: the path HR-07 → FS-02 → DC-01, the timeline, each step's technique and stage; click a step to see its log line.
6. **3:00 Look-ahead.** "This account has admin rights on 3 more servers", with the reason.
7. **3:30 Report and rules.** The plain-English report with citations (B-2); add a Sigma rule to `rules/` and re-run, no code change (B-4).
8. **4:15 Close.** "Whole dataset, one laptop, every step labelled and linked, every claim backed by a log line. Built for Mitigata's SOC and incident response."

Likely questions

Question	Answer
Doesn't Defender or CrowdStrike already do this?	Inside their own agent's data, yes. We work across vendors and on log dumps where no EDR was installed, which is the MSSP and incident-response reality.
Isn't this just a SIEM?	No SIEM is needed. We take raw log files and do detection, correlation and investigation together; if a SIEM exists, its alerts can be an extra input.
Can the LLM make things up?	It only narrates a graph built by code, and a plain-code checker drops any sentence without matching evidence. Grounding rate is on the results slide.
Can an attacker fool the AI through the logs?	We planted that attack in Scenario B. Log text is passed as data, flagged when it looks like an instruction, and the LLM cannot change the graph or verdict.

Question	Answer
What if logs are missing?	The engine shows a gap and marks links across it as inferred. It never fills a gap by guessing.
How do you avoid pulling in normal admin activity?	Links need evidence (process GUIDs, logon IDs, source IPs), not just a similar technique. The excluded list shows what was left out and why.
Is a lab realistic?	It runs real Windows, AD, Sysmon and auditd, attacked with MITRE Caldera, and is handed over as a plain log bundle, with normal activity mixed in. We also test on OTRF's independent recordings.
Does it scale?	Logs are parsed once into Parquet and queried with DuckDB by host and time; only the top-ranked clusters are traced, and the LLM sees only the pruned story, so LLM cost tracks incident size, not log volume.
Privacy: do logs go to an LLM vendor?	Only the pruned story, with usernames maskable; a local model can replace the API.
Why does Mitigata care?	Its SOC triages incidents at scale, and it sells incident response and cyber insurance; this speeds investigation and produces the report all three need.

Risks and fallbacks

The unknown dataset format and the time and memory targets are the biggest risks, so the pipeline is benchmarked at full size on synthetic data before kickoff.

Risk	Fallback
Dev sample formats differ from what we prepared	Format sniffing + a generic field mapper; all hands on parsers for the first two hours
Parsing misses the 1-hour budget	Parallel byte-range parsing; move the hot parser to Go/Rust or DuckDB native readers; stay on the Core tier

Risk	Fallback
Peak memory over 8 GB	DuckDB `memory_limit` with spill; smaller batches; graph only over hits
Accuracy drops on a bigger tier	The accuracy gate removes the multiplier, so stay on Core unless synthetic 30 GB holds accuracy
Too many false incidents	Raise the score threshold; require at least two linked tactics per incident; tune on clean synthetic sets
Planted steps missed (detection under 90%)	Tracer pulls in un-flagged events around each incident; add rarity features; review misses on the dev sample
Wrong technique labels	Parent technique as primary; one table of rules, reviewed against the dev sample
Look-ahead empty because privileges are rarely logged	Fall back to observed logons and admin-protocol connections; label as observed reach
LLM slow or unavailable	Reports are offline and optional; template report needs no LLM
Judges cannot start it	Docker Compose, a one-command README, a small sample bundle included for a smoke test

Sources

- Kotak Neo: SEBI fines CDSL ₹1 crore over the 2022 malware attack (Jul 2026)
- iThome: GOAD lab variants and resource needs
- OWASP Vulnerable Web Applications Directory: Game of Active Directory
- Wazuh documentation: MITRE ATT&CK module and Sysmon ingestion
- SIEM-Lab: Wazuh + Sysmon + ATT&CK detection lab write-up
- MITRE Caldera: operation results and report format
- OTRF Security-Datasets via MSTICPy
- Hayabusa (Yamato Security)
- CyberClear: benchmark for LLM attack-chain provenance (2026)